Monte Carlo Optimization

1 Smart Optimizations (Sequential Code)

The initial Monte Carlo pricer was correct, but slow. The issue was not the model itself, but how much unnecessary work was done inside the Monte Carlo loop.

The following optimizations were applied:

- **Memory reuse**: temporary vectors are reused instead of being reallocated millions of times. This avoids heavy memory overhead, like reusing the same notebook instead of buying a new one every time.
- **Invariant computations outside the loop**: time steps, square roots and drift terms do not depend on the simulated path. Computing them once instead of repeatedly removes a large amount of useless work.
- **Minimal inner loop**: the core loop only performs the stochastic update strictly required by the model.

These changes preserve the Monte Carlo method and its numerical precision. The sequential execution time was reduced by roughly **a factor of 3 to 4**.

2 CUDA, MPI or OpenMP?

Several parallel approaches were considered:

- **CUDA**: GPUs offer massive parallelism but require a full redesign. Think of thousands of very fast but very simple workers. Powerful, but unnecessary here.
- **MPI**: MPI targets distributed systems. It can be seen as a team of researchers in different cities constantly exchanging emails. Effective at scale, but overkill on a single machine.
- **OpenMP**: OpenMP uses multiple CPU cores sharing the same memory. More like a small group of smart people working in the same room. Simple, efficient, and perfectly suited to Monte Carlo paths.

Given the problem and the hardware, **OpenMP was the obvious choice**.

3 Applying OpenMP

Monte Carlo paths are independent, which makes the algorithm embarrassingly parallel.

Before parallelization:

- Temporary buffers were made `thread_local` to ensure thread safety.
- Partial sums were accumulated locally and combined safely.
- Only the loop over paths was parallelized to avoid unnecessary synchronization.

The OpenMP directive was then applied directly to the Monte Carlo loop. All performance measurements were done in **Release mode**, since Debug mode gives misleading results.

4 Results

Figure 1 shows the execution times obtained with the optimized sequential version. Even after removing unnecessary overhead, pricing complex products such as Asian options still requires tens of seconds for tens of millions of Monte Carlo paths.

Figure 2 presents the results obtained after applying OpenMP parallelization. The reduction in execution time is immediate and significant: the same number of Monte Carlo paths is processed in only a few seconds.

Depending on the option type, speedups between **6× and 10×** are observed. The largest gains appear for computationally intensive products, such as Asian options, which benefit the most from parallel execution.

The estimated prices remain statistically consistent with the sequential version and stay well within confidence intervals, confirming that no numerical approximation was introduced.

5 Conclusion

By cleaning up the sequential code and applying OpenMP parallelization, the Monte Carlo pricer becomes fast and scalable. The final implementation remains simple, precise, and well suited for modern multi-core CPUs.

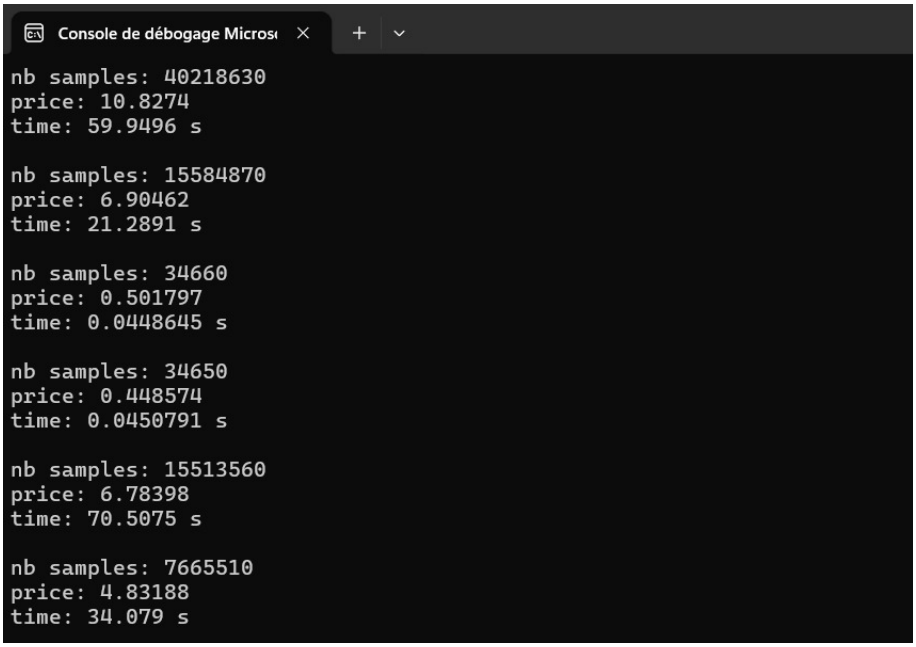

Figure 1: Execution times for the optimized sequential Monte Carlo pricer.

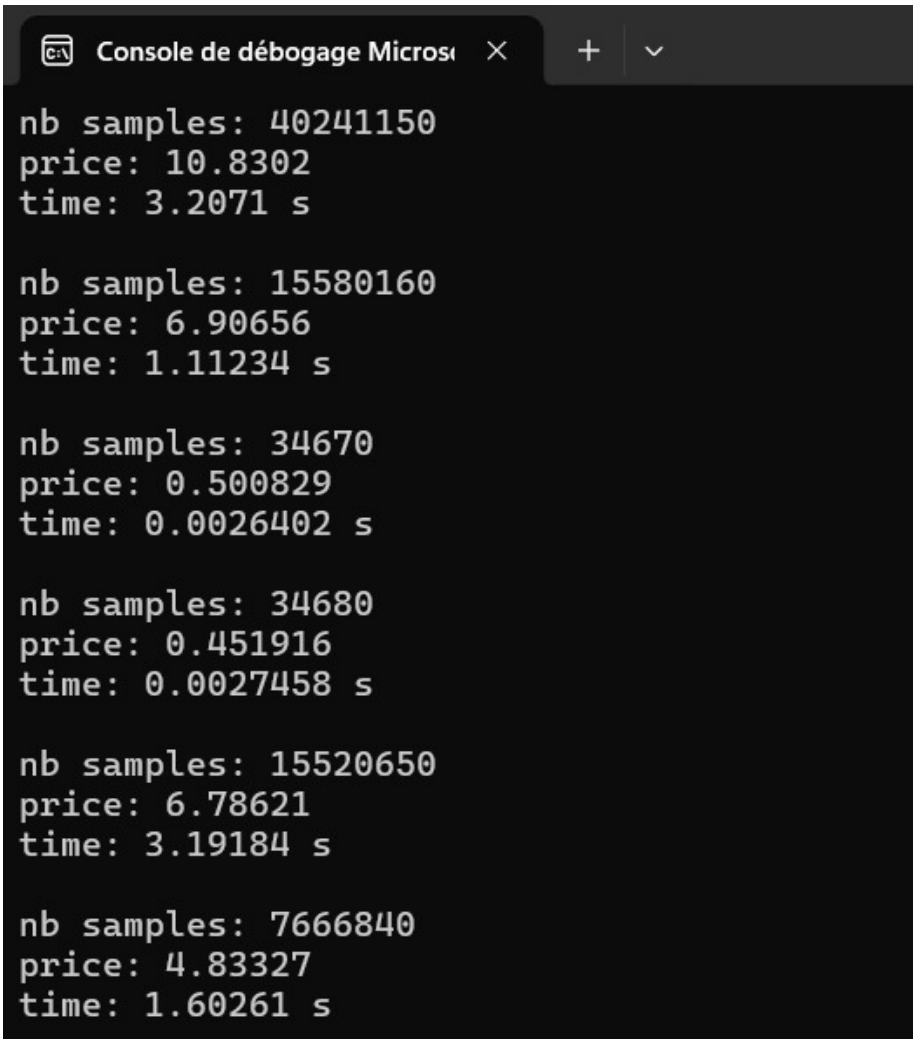

Figure 2: Same execution times for the OpenMP parallel Monte Carlo pricer.